Struct vs Class

Absolutely. For OOP interviews, `struct` vs `class` is a **small topic but a very common C++ interview question**. The key is understanding that they are almost identical, with one important default difference.

`struct` vs `class` in C++

1. The core difference

In C++, **both `struct` and `class` can have:**

- Data members
- Member functions
- Constructors/destructors
- Access specifiers
- Inheritance
- Virtual functions
- Static members
- Templates
- etc.

The main difference is their **default access control**.

Feature`structclass`Default member access`publicprivate`Default inheritance`publicprivate`Can have methods?YesYesCan have constructors?YesYesCan use inheritance?YesYesCan have virtual functions?YesYes

That's basically the interview answer.

---

2. Default member access

`struct`

```
struct Student {
    int age;
};

int main() {
    Student s;
    s.age = 20;   // ☑ public by default
}
```

Because members of a `struct` are **public by default**.

Equivalent to:

```
struct Student {
public:
    int age;
};
```

`class`

```
class Student {
    int age;
};

int main() {
    Student s;
    s.age = 20;   // ❌ Error: private
}
```

Because members of a `class` are **private by default**.

Equivalent to:

```
class Student {
private:
    int age;
};
```

So remember:

> **struct → public by default**

> **class → private by default**

3. But both can explicitly use access specifiers

This is important.

You can absolutely do:

```
struct Student {
private:
    int age;

public:
    void setAge(int x) {
        age = x;
    }
};
```

And:

```
class Student {
public:
    int age;
};
```

So `struct` does **NOT** mean "no encapsulation."

Both support encapsulation.

---

4. Default inheritance is also different

This is the second difference interviewers sometimes ask.

Suppose:

```
class Animal {
public:
    void eat() {}
};
```

Now:

Class inheritance

```
class Dog : Animal {
};
```

Inheritance is **private by default**.

So this is equivalent to:

```
class Dog : private Animal {
};
```

Struct inheritance

```
struct Dog : Animal {
};
```

Inheritance is **public by default**.

Equivalent to:

```
struct Dog : public Animal {
};
```

Therefore:

> `struct` → public default inheritance

> `class` → private default inheritance

---

5. Why does C++ have both?

This is more of a **design convention** than a technical necessity.

Use `struct` when:

You're representing a simple collection of data.

```cpp
struct Point {
    int x;
    int y;
};
```

You generally expect:

```cpp
Point p;
p.x = 10;
p.y = 20;
```

The data is naturally public.

Use `class` when:

You're modeling an object with **behavior + encapsulated state**.

```cpp
class BankAccount {
private:
    double balance;

public:
    void deposit(double amount) {
        balance += amount;
    }

    double getBalance() {
        return balance;
    }
};
```

Here you don't want:

```cpp
account.balance = -100000;
```

Instead, you control access through methods.

---

6. Very important misconception

A common beginner misconception:

> "`struct` is for data and `class` is for functions."

❌ **Wrong.**

A `struct` can have everything a class has:

```cpp
struct Animal {
private:
    int age;

public:
    Animal(int a) {
        age = a;
    }

    void speak() {
        cout << "Animal speaks";
    }

    virtual void sound() {
        cout << "Some sound";
    }
};
```

This is completely valid C++.

Similarly, a class can just contain data:

```cpp
class Point {
public:
    int x;
    int y;
};
```

So the **real language-level difference is defaults**, not capability.

---

7. Interview mental model

Think:

```
struct
   ↓
"Everything public unless I say otherwise"

class
   ↓
"Everything private unless I say otherwise"
```

And for inheritance:

```
struct Derived : Base
        ↓
public inheritance

class Derived : Base
        ↓
private inheritance
```

---

8. Common interview questions

Q1. What is the difference between struct and class in C++?

Ideal answer:

> In C++, `struct` and `class` are almost identical in functionality. The main difference is that members of a `struct` are public by default, while members of a `class` are private by default. Similarly, inheritance from a `struct` is public by default, whereas inheritance from a `class` is private by default.

That's a strong interview answer.

Q2. Can a struct have functions?

Yes.

```cpp
struct Student {
    int age;

    void display() {
        cout << age;
    }
};
```

Q3. Can a struct have a constructor?

Yes.

```cpp
struct Student {
    int age;

    Student(int a) {
        age = a;
    }
};
```

Q4. Can a struct inherit from a class?

Yes.

```
class Animal {
public:
    void eat() {}
};

struct Dog : Animal {
};
```

And because `Dog` is a `struct`, inheritance is public by default.

Q5. Can a class inherit from a struct?

Yes.

```
struct Animal {
    void eat() {}
};

class Dog : Animal {
};
```

But inheritance is **private by default** because `Dog` is a class.

Q6. Can a struct have private members?

Yes.

```
struct Student {
private:
    int age;

public:
    void setAge(int a) {
        age = a;
    }
};
```

Q7. Is a struct stored differently from a class?

No.

There is no inherent memory-layout/performance difference simply because one is a `struct` and the other is a `class`.

For example:

```
struct A {
    int x;
};

class B {
public:
    int x;
};
```

They have essentially the same object representation under the same relevant layout conditions.

---

9. One tricky interview question

What does this mean?

```
class Dog : Animal {
};
```

Many people answer:

> Dog inherits Animal.

That's incomplete.

The correct answer is:

> `Dog` privately inherits from `Animal` because `Dog` is a `class` and the inheritance access is omitted.

Whereas:

```
struct Dog : Animal {
};
```

means:

> `Dog` publicly inherits from `Animal`.

---

10. One-line cheat sheet

```
struct vs class

struct:
    default members     → public
    default inheritance → public

class:
    default members     → private
    default inheritance → private

Otherwise:
    Both support constructors, methods, inheritance,
    polymorphism, virtual functions, encapsulation, etc.
```

Interview priority

For your OOP prep, remember these **3 things**:

1. **Both are almost functionally identical.**
2. `struct` → **public by default,** `class` → **private by default.**
3. `struct` **inheritance → public by default,** `class` **inheritance → private by default.**

Next, a very important concept to connect this with is `public` **vs** `private` **vs** `protected`, because that will make inheritance and encapsulation much easier to understand.